

Firewall iptables entre Kali, Ubuntu y Windows10

6 reglas de filtrado, pruebas de tráfico y verificación con Wireshark — Seguridad de la Información

1. Objetivo

Configurar Ubuntu como firewall (iptables, cadena FORWARD) entre dos máquinas virtuales, Kali y Windows10, implementando 6 reglas de filtrado -- 3 dirigidas al tráfico de Windows10 y 3 al de Kali -- y verificar cada una generando tráfico real y capturando la evidencia con Wireshark: direcciones IP, puertos, protocolo y banderas TCP/tipo ICMP correspondientes, comparando el comportamiento permitido, bloqueado y rechazado.

2. Topología de red

Se reutiliza la topología del laboratorio reducido de 3 VMs: Kali y Windows10 quedan en subredes host-only distintas, y todo el tráfico entre ellas atraviesa obligatoriamente a Ubuntu, que decide con iptables qué se permite.

VM	Rol	IP	Red host-only
Kali	Extremo de red	192.168.50.10	DMZ Externa (192.168.50.0/24)
Ubuntu	Firewall (iptables, FORWARD)	192.168.50.2 / 192.168.60.2	Ambas (dos interfaces)
Windows10	Extremo de red	192.168.60.10	Zona Empresarial (192.168.60.0/24)

Nota: la IP de Ubuntu en cada red es .2 y no .1, porque .1 ya está asignada al propio adaptador Host-Only del equipo físico (VirtualBox); usar la misma IP para Ubuntu habría generado un conflicto de direcciones.

3. Reglas de firewall implementadas

Las 6 reglas se cargan en la cadena FORWARD de Ubuntu, con política por defecto DROP y una excepción inicial para el tráfico de respuesta de conexiones ya establecidas:

```
sudo iptables -F FORWARD
sudo iptables -P FORWARD DROP

sudo iptables -A FORWARD -m state --state ESTABLISHED,RELATED -j ACCEPT

# ---- hacia WINDOWS10 (192.168.60.10) ----
sudo iptables -A FORWARD -p tcp -d 192.168.60.10 --dport 445 -j DROP
# W1
sudo iptables -A FORWARD -p icmp -d 192.168.60.10 --icmp-type echo-request -j DROP
# W2
sudo iptables -A FORWARD -p tcp -d 192.168.60.10 --dport 8080 -j ACCEPT
# W3

# ---- hacia KALI (192.168.50.10) ----
sudo iptables -A FORWARD -p tcp -d 192.168.50.10 --dport 22 -j REJECT --reject-with
tcp-reset # K1
sudo iptables -A FORWARD -p udp -d 192.168.50.10 --dport 53 -j REJECT --reject-with
icmp-port-unreachable # K2
sudo iptables -A FORWARD -p icmp -d 192.168.50.10 --icmp-type echo-request -j ACCEPT
# K3
```

#	Tráfico hacia	Protocolo / puerto	Acción	Justificación
W1	Windows10 (.60.10)	TCP / 445 (SMB)	DROP	Puerto de archivos compartidos, no debe ser alcanzable desde otra zona
W2	Windows10 (.60.10)	ICMP echo-request	DROP	Ping bloqueado: el host queda invisible a barridos de red
W3	Windows10 (.60.10)	TCP / 8080	ACCEPT	Único servicio que Windows10 expone hacia afuera
K1	Kali (.50.10)	TCP / 22 (SSH)	REJECT (tcp-reset)	Gestión de Kali no debe alcanzarse desde la otra zona
K2	Kali (.50.10)	UDP / 53 (DNS)	REJECT (icmp-port-unreachable)	Kali no es un servidor DNS: puerto cerrado por política
K3	Kali (.50.10)	ICMP echo-request	ACCEPT	Monitoreo básico de disponibilidad permitido

Balance intencional: 2 reglas ACCEPT, 2 DROP y 2 REJECT -- cubre los tres comportamientos que pide comparar el taller: tráfico permitido, bloqueado (silencioso) y rechazado (con respuesta activa).

4. Metodología de pruebas

El tráfico de las reglas W1-W3 (dirigidas a Windows10) se genera desde Kali, siguiendo lo indicado en el enunciado. El tráfico de las reglas K1-K3 (dirigidas a Kali) se genera desde Windows10, porque son reglas que filtran tráfico con destino Kali -- no puede probarse generando tráfico desde el propio Kali. La captura de paquetes se realizó en Ubuntu (interfaz "any"), por ser el único punto que ve las dos caras de cada prueba: el paquete entrando por una interfaz y si sale -- o no -- por la otra.

5. Resultados por regla

5.1 -- W1: bloquear TCP/445 hacia Windows10 (DROP)

Comando ejecutado:

```
nmap -p 445 -sT -Pn 192.168.60.10
```

Resultado obtenido:

```
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-23 23:55 -0500
Nmap scan report for 192.168.60.10
Host is up.

PORT      STATE      SERVICE
445/tcp   filtered  microsoft-ds

Nmap done: 1 IP address (1 host up) scanned in 2.53 seconds
```

No.	Time	Source	Destination	Protocol	Length	Info
17	0.7271257	192.168.50.10	192.168.60.10	TCP	76	49588 → 445 [CN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1324078030 TSecr=0 WS=512
18	0.8049255	192.168.60.10	192.168.60.10	TCP	76	445 → 49588 [ACK] Seq=0 Win=0 Len=0 MSS=1460 SACK_PERM TSval=1324078030 TSecr=0 WS=512
19	40.975211648	192.168.50.10	192.168.60.10	TCP	76	49588 → 445 [CN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=1324078030 TSecr=0 WS=512

Evidencia Wireshark — filtro: `ip.addr == 192.168.60.10 && tcp.port == 445`

5.2 -- W2: bloquear ICMP (ping) hacia Windows10 (DROP)

Comando ejecutado:

```
ping -c 4 192.168.60.10
```

Resultado obtenido:

```
PING 192.168.60.10 (192.168.60.10) 56(84) bytes of data.
--- 192.168.60.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3067ms
```

No.	Time	Source	Destination	Protocol	Length	Info
20	41.886938856	192.168.50.10	192.168.60.10	ICMP	100	Echo (ping) request id=0x7bcc, seq=1/256, ttl=64 (no response found)
21	42.891820868	192.168.50.10	192.168.60.10	ICMP	100	Echo (ping) request id=0x7bcc, seq=2/512, ttl=64 (no response found)
22	43.115122985	192.168.50.10	192.168.60.10	ICMP	100	Echo (ping) request id=0x7bcc, seq=3/768, ttl=64 (no response found)
23	44.139468575	192.168.50.10	192.168.60.10	ICMP	100	Echo (ping) request id=0x7bcc, seq=4/1024, ttl=64 (no response found)

Evidencia Wireshark — filtro: `ip.src == 192.168.50.10 && ip.dst == 192.168.60.10 && icmp.type == 8`

5.3 -- W3: permitir solo TCP/8080 hacia Windows10 (ACCEPT)

Comando ejecutado:

```
curl -v http://192.168.60.10:8080
```

Resultado obtenido:

```

* Trying 192.168.60.10:8080...
* Established connection to 192.168.60.10 (192.168.60.10 port 8080) from 192.168.50.10
port 38492
* using HTTP/1.x
> GET / HTTP/1.1
> Host: 192.168.60.10:8080
> User-Agent: curl/8.20.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Transfer-Encoding: chunked
< Server: Microsoft-HTTPAPI/2.0
< Date: Mon, 24 Aug 2026 05:02:22 GMT
<
* Connection #0 to host 192.168.60.10:8080 left intact

```

No.	Time	Source	Destination	Protocol	Length	Info
41	0.000000	192.168.60.10	192.168.50.10	TCP	76	TCP [RST] Seq=8080 Win=0 Len=0 (RST) Seq=8080 Win=0 Len=0
42	0.000000	192.168.50.10	192.168.60.10	TCP	76	TCP [RST] Seq=8080 Win=0 Len=0 (RST) Seq=8080 Win=0 Len=0
43	0.000000	192.168.50.10	192.168.60.10	TCP	76	TCP [RST] Seq=8080 Win=0 Len=0 (RST) Seq=8080 Win=0 Len=0
44	0.000000	192.168.50.10	192.168.60.10	TCP	76	TCP [RST] Seq=8080 Win=0 Len=0 (RST) Seq=8080 Win=0 Len=0
45	0.000000	192.168.50.10	192.168.60.10	TCP	76	TCP [RST] Seq=8080 Win=0 Len=0 (RST) Seq=8080 Win=0 Len=0
46	0.000000	192.168.50.10	192.168.60.10	TCP	76	TCP [RST] Seq=8080 Win=0 Len=0 (RST) Seq=8080 Win=0 Len=0
47	0.000000	192.168.50.10	192.168.60.10	HTTP	138	GET / HTTP/1.1
48	0.000000	192.168.60.10	192.168.50.10	HTTP	138	HTTP/1.1 200 OK
49	0.000000	192.168.50.10	192.168.60.10	TCP	76	TCP [RST] Seq=8080 Win=0 Len=0 (RST) Seq=8080 Win=0 Len=0
50	0.000000	192.168.50.10	192.168.60.10	TCP	76	TCP [RST] Seq=8080 Win=0 Len=0 (RST) Seq=8080 Win=0 Len=0
51	0.000000	192.168.50.10	192.168.60.10	TCP	76	TCP [RST] Seq=8080 Win=0 Len=0 (RST) Seq=8080 Win=0 Len=0
52	0.000000	192.168.50.10	192.168.60.10	TCP	76	TCP [RST] Seq=8080 Win=0 Len=0 (RST) Seq=8080 Win=0 Len=0
53	0.000000	192.168.50.10	192.168.60.10	TCP	76	TCP [RST] Seq=8080 Win=0 Len=0 (RST) Seq=8080 Win=0 Len=0
54	0.000000	192.168.50.10	192.168.60.10	TCP	76	TCP [RST] Seq=8080 Win=0 Len=0 (RST) Seq=8080 Win=0 Len=0
55	0.000000	192.168.50.10	192.168.60.10	TCP	76	TCP [RST] Seq=8080 Win=0 Len=0 (RST) Seq=8080 Win=0 Len=0
56	0.000000	192.168.50.10	192.168.60.10	TCP	76	TCP [RST] Seq=8080 Win=0 Len=0 (RST) Seq=8080 Win=0 Len=0
57	0.000000	192.168.50.10	192.168.60.10	TCP	76	TCP [RST] Seq=8080 Win=0 Len=0 (RST) Seq=8080 Win=0 Len=0
58	0.000000	192.168.50.10	192.168.60.10	TCP	76	TCP [RST] Seq=8080 Win=0 Len=0 (RST) Seq=8080 Win=0 Len=0

Evidencia Wireshark — filtro: `ip.addr == 192.168.60.10 && tcp.port == 8080`

5.4 -- K1: bloquear TCP/22 hacia Kali (REJECT, tcp-reset)

Comando ejecutado:

```
Test-NetConnection -ComputerName 192.168.50.10 -Port 22
```

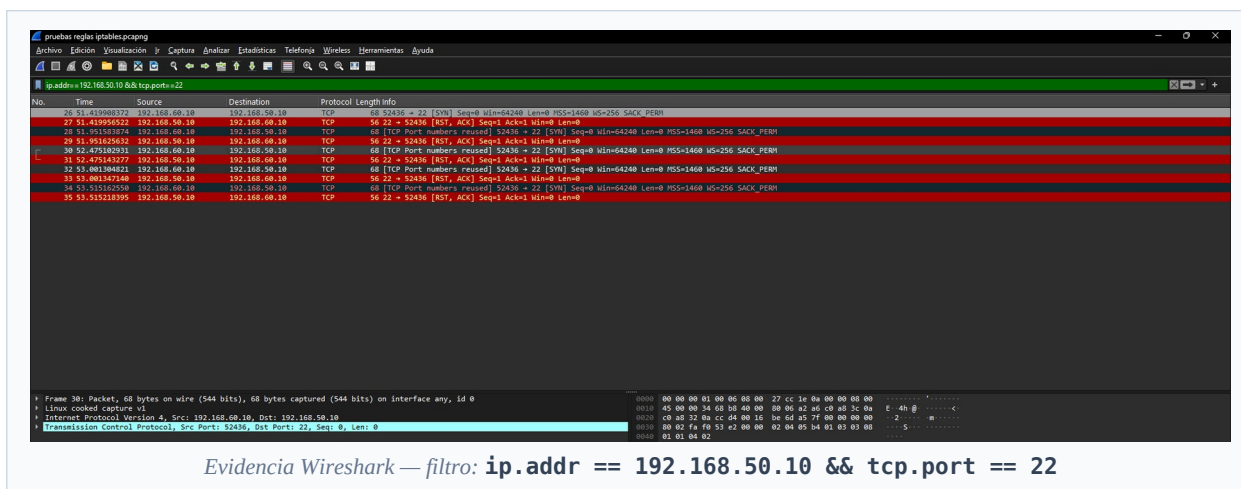
Resultado obtenido:

```
WARNING: TCP connect to (192.168.50.10 : 22) failed
```

```

ComputerName      : 192.168.50.10
RemoteAddress     : 192.168.50.10
RemotePort        : 22
InterfaceAlias    : Ethernet
SourceAddress     : 192.168.60.10
PingSucceeded     : True
PingReplyDetails (RTT) : 2 ms
TcpTestSucceeded  : False

```



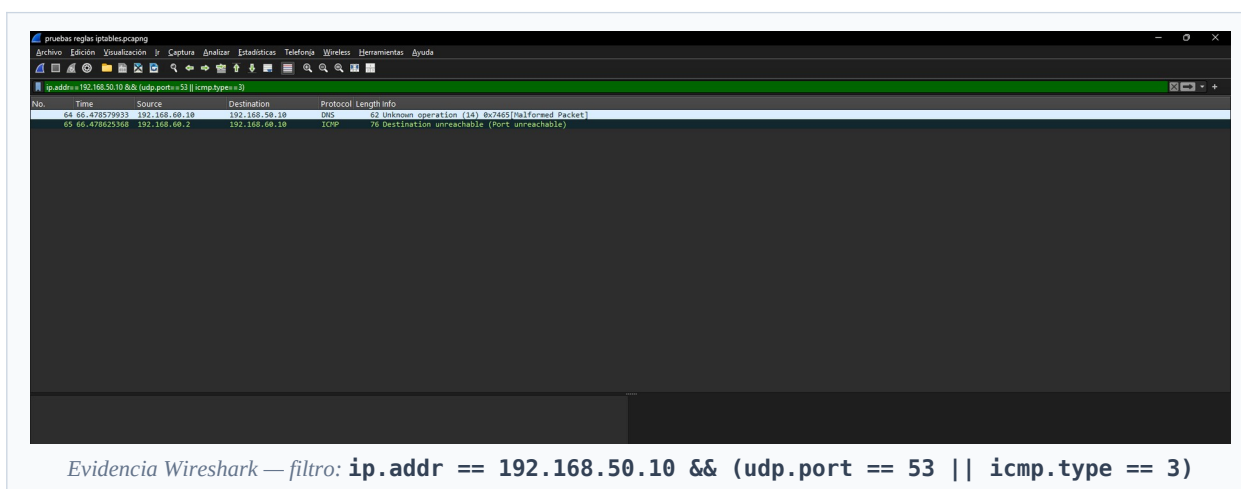
5.5 -- K2: bloquear UDP/53 hacia Kali (REJECT, icmp-port-unreachable)

Comando ejecutado:

```
$udp = New-Object System.Net.Sockets.UdpClient
$udp.Client.ReceiveTimeout = 3000
$udp.Connect("192.168.50.10", 53)
$bytes = [System.Text.Encoding]:ASCII.GetBytes("test")
$udp.Send($bytes, $bytes.Length) | Out-Null
try { $udp.Receive([ref]$null); "Respuesta recibida" }
catch [System.Net.Sockets.SocketException] {
    "SocketException: $($_.Exception.SocketErrorCode) - $($_.Exception.Message)"
}
```

Resultado obtenido:

SocketException: ConnectionReset - An existing connection was forcibly closed by the remote host



5.6 -- K3: permitir ICMP (ping) hacia Kali (ACCEPT)

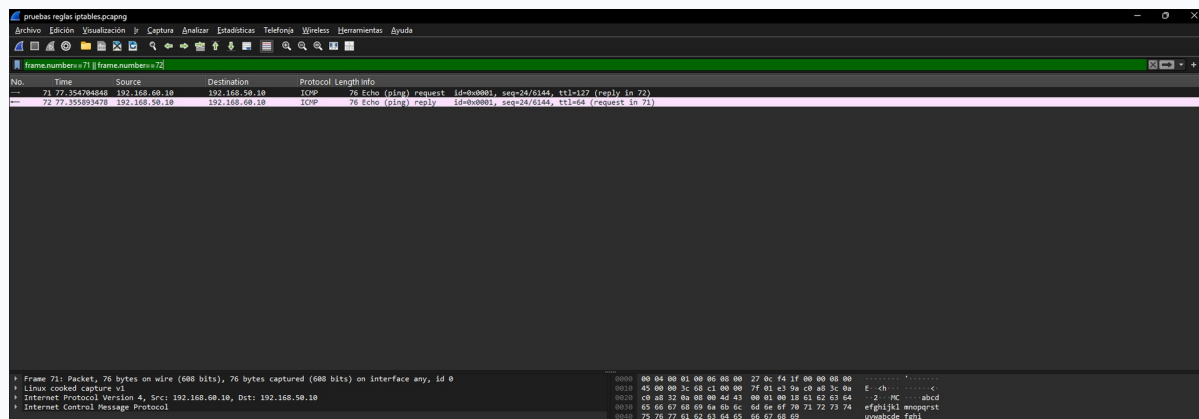
Comando ejecutado:

```
ping 192.168.50.10 (verificado también como componente de la prueba K1:
PingSucceeded=True, RTT=2 ms)
```

Resultado obtenido:

Confirmado por dos vías:

- 1) El campo "PingSucceeded: True / RTT: 2 ms" de la prueba K1 (mismo destino, Kali).
- 2) Los contadores del firewall: la regla 7 (ACCEPT icmp hacia Kali) acumuló paquetes en cada ronda de pruebas, y la regla 1 (ESTABLISHED,RELATED) subió en paralelo -- confirma que la respuesta (echo-reply) de Kali vuelve correctamente hacia Windows10.



Evidencia Wireshark — filtro: `frame.number == 71 || frame.number == 72` (aislando un par request/reply de icmp && ip.addr == 192.168.50.10 && ip.addr == 192.168.60.10)

6. Verificación de aislamiento cruzado

Con las 6 reglas cargadas simultáneamente, se confirma que las reglas de Windows10 no afectan el tráfico de Kali y viceversa:

```
# desde Kali -- W3 debe seguir funcionando aunque ya estén cargadas las reglas de Kali (K1/K2/K3)
curl http://192.168.60.10:8080

# desde Windows10 -- K1 debe seguir funcionando aunque ya estén cargadas las reglas de Windows (W1/W2/W3)
Test-NetConnection -ComputerName 192.168.50.10 -Port 22
```

W3 (Kali -> Windows10:8080) y K1 (Windows10 -> Kali:22) se comportaron exactamente igual que en sus pruebas individuales de las secciones 5.3 y 5.4 -- el aislamiento entre los dos grupos de reglas queda confirmado.

7. Evidencia -- contadores finales del firewall

Salida de `sudo iptables -L FORWARD -v -n --line-numbers` tomada después de ejecutar todas las pruebas, confirmando con paquetes reales que cada regla se disparó:

```
Chain FORWARD (policy DROP 6 packets, 248 bytes)
num  pkts bytes target    prot opt in     out     source destination
1      40  2616 ACCEPT          all  --  *      *       0.0.0.0/0  0.0.0.0/0
state RELATED,ESTABLISHED
2       2   120 DROP          tcp  --  *      *       0.0.0.0/0  192.168.60.10
tcp dpt:445
3       6   392 DROP          icmp --  *      *       0.0.0.0/0  192.168.60.10
icmp type 8
4      25  1500 ACCEPT          tcp  --  *      *       0.0.0.0/0  192.168.60.10
tcp dpt:8080
5       5   260 REJECT          tcp  --  *      *       0.0.0.0/0  192.168.50.10
tcp dpt:22 reject-with tcp-reset
6       3    96 REJECT          udp  --  *      *       0.0.0.0/0  192.168.50.10
udp dpt:53 reject-with icmp-port-unreachable
7      14   840 ACCEPT          icmp --  *      *       0.0.0.0/0  192.168.50.10
icmp type 8
```

La política *DROP por defecto* también acumuló paquetes (6), correspondientes a tráfico que no coincidió con ninguna de las 6 reglas explícitas durante las distintas rondas de prueba.

8. Conclusiones

- **DROP (W1, W2):** dejan pasar el tráfico sin ninguna señal de respuesta: el origen solo ve un timeout. Es el comportamiento correcto para "esconder" un servicio.
- **REJECT (K1, K2):** generan una respuesta activa e inmediata (RST para TCP, ICMP port-unreachable para UDP) -- el origen sabe de inmediato que fue rechazado, en vez de esperar un timeout.
- **ACCEPT (W3, K3):** permiten el tráfico de punta a punta; se verificó con el handshake TCP completo y con el intercambio ICMP request/reply.
- **Defensa en capas:** aparte de las reglas en Ubuntu, fue necesario agregar excepciones puntuales en el Firewall de Windows (puerto TCP 8080 y el tipo ICMP 3) para que el tráfico ya permitido por Ubuntu llegara hasta la aplicación -- evidencia de que la red y el host filtran de forma independiente, y ambas capas deben coincidir en dejar pasar el tráfico.